

Rank Ordered Logit from Scratch

2026-07-26

Purpose of this document

The purpose of this document is to present the mathematics underlying the Rank Ordered Logit of Beggs, Cardell, and Hausman (1981), and to show how every quantity involved (the log-likelihood, the gradient, the Hessian, the maximum likelihood estimates, their standard errors, and various post-estimation quantities such as counterfactual probabilities and the log-sum) can be computed and estimated using only base R, without relying on the `apollo` package.

The Rank Ordered Logit (ROL)

Researchers often create surveys where respondents are asked to rank some alternatives. This could be stated preferences over transportation modes, brands, etc. The ROL model is the standard way to analyze preferences in such context.

The setup is the following: there are $j = 1, \dots, J$ alternatives that are ranked by $i = 1, \dots, N$ respondents. These alternatives are associated with respondent utilities U_{ij} , $\forall i, j$.

Because preferences are assumed to be known by the respondents but are not known the researcher, the random utility framework of Manski (1977) is used. The specification for the random utilities is the following:

$$U_{ij} = V_{ij} + \varepsilon_{ij},$$

where it is traditionally assumed that V_{ij} takes a linear index form $\mathbf{x}'_{ij}\boldsymbol{\beta}$, where $\mathbf{x}_{ij}$ is a K-vector of observed characteristics that may vary across individuals and alternatives. $\boldsymbol{\beta}$ is K-vector of parameters to be estimated.

First, suppose the survey only asks about the preferred alternative of individuals. Assuming that the ε_{ij} 's terms are i.i.d EV1, then the probability that alternative j is the preferred by individual i takes the standard logit probability form:

$$\mathbb{P}(U_{ij} > U_{ik} \forall k \neq j) = \frac{\exp(V_{ij})}{\sum_{l=1}^J \exp(V_{il})}$$

When all alternatives are ranked by the respondents, the model should take this into account in order to achieve an efficiency gain. Denote by r_{ij} the variety receiving rank j by individual i , and store it in the vector of rankings $\mathbf{r}_i = (r_{i1}, \dots, r_{iJ})'$ for all individuals i . The probability of observing such a ranking in the ROL model is given by

$$\mathbb{P}(U_{ir_{i1}} > U_{ir_{i2}} > \dots > U_{ir_{iJ}}) = \frac{\exp(V_{ir_{i1}})}{\sum_{l=1}^J \exp(V_{ir_{il}})} \times \frac{\exp(V_{ir_{i2}})}{\sum_{l=2}^J \exp(V_{ir_{il}})} \times \dots \times \frac{\exp(V_{ir_{i,J-1}})}{\sum_{l=J-1}^J \exp(V_{ir_{il}})} = \prod_{j=1}^{J-1} \frac{\exp(V_{ir_{ij}})}{\sum_{l=j}^J \exp(V_{ir_{il}})}.$$

The above equation is the contribution of respondent i to the the likelihood. For an independent sample of respondents, we therefore obtain the following likelihood function (using the expression for $V_{ir_{ij}}$):

$$\mathcal{L}(\beta) = \prod_{i=1}^N \prod_{j=1}^{J-1} \frac{\exp(\mathbf{x}'_{ir_{ij}}\beta)}{\sum_{l=j}^J \exp(\mathbf{x}'_{ir_{il}}\beta)}.$$

In that case, the associated log-likelihood writes:

$$\ell(\beta) = \sum_{i=1}^N \sum_{j=1}^{J-1} \mathbf{x}'_{ir_{ij}}\beta - \sum_{i=1}^N \sum_{j=1}^{J-1} \log \left(\sum_{l=j}^J \exp(\mathbf{x}'_{ir_{il}}\beta) \right)$$

This function is globally concave in β , which guarantees that any local maximum found by a Newton-type algorithm is in fact the unique global maximum.

Obtaining the associated gradient is straightforward:

$$\nabla_{\beta} \ell(\beta) = \sum_{i=1}^N \sum_{j=1}^{J-1} \left(\mathbf{x}_{ir_{ij}} - \sum_{l=j}^J P_{ir_{il}|j} \mathbf{x}_{ir_{il}} \right),$$

where $P_{ir_{il}|j} = \frac{\exp(\mathbf{x}'_{ir_{il}}\beta)}{\sum_{k=j}^J \exp(\mathbf{x}'_{ir_{ik}}\beta)}$ is the conditional probability for respondent i that the alternative ranked at the l^{th} position (where $l \geq j$) is chosen as the “best” among the subset of alternatives that remain at stage j of the ranking process.

As our objective is to find an estimate $\hat{\beta}$ setting the gradient equal to zero, this tells us that the Maximum Likelihood Estimate is such that, for each rank position, the sample average of each covariate among the alternatives that received that rank must equal the average of that covariate predicted by the model over the alternatives that remain at that stage.

Again, it is straightforward to show that the K by K Hessian matrix has the following expression:

$$\mathcal{H}(\beta) = -\nabla_{\beta\beta'} \ell(\beta) = \sum_{i=1}^N \sum_{j=1}^{J-1} \sum_{l=j}^J P_{ir_{il}|j} \left(\mathbf{x}_{ir_{il}} - \sum_{k=j}^J \mathbf{x}_{ir_{ik}} P_{ir_{ik}|j} \right) \mathbf{x}'_{ir_{il}}$$

Application to the drugChoice Data

We apply this model with a Monte Carlo simulation based on Fok, Paap, and Van Dijk (2010).

In their simulation setup, there are $N = 1000$ respondents ranking $J = 4$ alternatives. Importantly, it is assumed that these respondents are able to rank their most preferred alternatives but may have some issues when ranking the remaining ones. This assumption will show one of the limitations of the ROL model that we will discuss later on.

To capture this inability of ranking less desired alternatives, respondents are classed in segments. For instance, if an individual belongs to segment k , then she is able to rank correctly her k^{th} most preferred alternatives, but will rank randomly the remaining $J - k$ alternatives. In the simulation, respondents are classed into the segments with probabilities $p_0 = 0, p_1 = p_2 = 0.3, p_3 = 0.4$. The fact that $p_0 = 0$ means that all respondents are able to rank their most preferred item (this implicitly means that the parameters of the multinomial logit model can be estimated consistently).

The random utility is generated from the following DGP:

$$U_{ij} = \beta_{0j} + x_{i1}\beta_{1j} + x_{i2}\beta_{2j} + \epsilon_{ij}; \quad j = 1, \dots, 4; \quad i = 1, \dots, 1000; \quad \epsilon_{ij} \sim \text{EV1 (iid)},$$

where x_{i1} is standard normal and x_{i2} is a dummy equal to 1 with probability 1/2.

Because only differences in utility matter, the coefficients of one alternative need to be normalized. The fourth category is therefore taken as the reference with $\beta_{k4} = 0 \forall k = 0, 1, 2$.

The true values of β are $(\beta_{01}, \beta_{11}, \beta_{21}, \beta_{02}, \beta_{12}, \beta_{22}, \beta_{03}, \beta_{13}, \beta_{23})' = (1, 0.75, -0.3, 0.25, -0.5, 0.45, -0.25, 1.00, 0.8)$.

To compare the estimates from the ROL model with the true value of the parameters of interest, we generate as in the paper 10 000 Monte Carlo replications with this DGP and estimate the ROL model for each replication. We then look at the mean value of the estimated parameters across the replications and compare it with the true values.

The Newton-Raphson algorithm

$$\beta^{(t+1)} = \beta^{(t)} + [\mathcal{H}(\beta^{(t)})]^{-1} g(\beta^{(t)}).$$

The algorithm is initialized at $\beta^{(0)} = \mathbf{0}$ and iterates until the parameter update falls below a small tolerance threshold, i.e. $\max_k |\beta_k^{(t+1)} - \beta_k^{(t)}| < \text{tol}$. Because the ROL log-likelihood is globally concave and here we use the *exact* analytical gradient and Hessian (rather than numerical or quasi-Newton approximations), the algorithm typically converges in only a handful of iterations to the unique maximum likelihood estimate $\hat{\beta}$.

Variance matrix and standard errors

Under standard regularity conditions and a correctly specified model, the maximum likelihood estimator $\hat{\beta}$ is asymptotically normal with variance-covariance matrix equal to the inverse of the (expected) Fisher information matrix. This is estimated by:

$$\text{Var}(\hat{\beta}) \approx [\mathcal{H}(\hat{\beta})]^{-1}.$$

The (asymptotic) standard error of each estimated coefficient $\hat{\beta}_k$ is then obtained as the square root of the corresponding diagonal element of this variance-covariance matrix,

$$\text{se}(\hat{\beta}_k) = \sqrt{[\mathcal{H}(\hat{\beta})^{-1}]_{kk}}.$$

Application to the Fok, Paap and van Dijk (2010) Monte Carlo simulation

R code

```
## =====
## Application of the Rank Ordered Logit (ROL) model
##
## Data generation for the Monte Carlo simulation in:
## Fok, Paap & van Dijk (2010), "A Rank-Ordered Logit Model with
## Unobserved Heterogeneity in Ranking Capabilities", Section 3.
```

```

## =====

rm(list = ls())
set.seed(123)

## -----
## Fixed settings of the simulation
## -----

J <- 4      # number of alternatives
N <- 1000    # number of individuals (as in Table I of the paper)

## True beta parameters (Table I, "True" column).
## Category J = 4 is the base category, so beta_{.,4} = 0.
beta0 <- c(1.00, 0.25, -0.25, 0) # intercepts,      j = 1,2,3,4
beta1 <- c(0.75, -0.50, 1.00, 0) # coefficient x1,   j = 1,2,3,4
beta2 <- c(-0.30, 0.45, 0.80, 0) # coefficient x2,   j = 1,2,3,4

## Segment (ranking-capability) probabilities.
## Segment k = 0,...,J-1 : individual can correctly rank the k
## most-preferred items; the remaining J-k items are ranked randomly.
## p0 = 0 : nobody is completely unable to rank (as chosen in the paper).
p_seg <- c(p0 = 0.0, p1 = 0.3, p2 = 0.3, p3 = 0.4)
stopifnot(abs(sum(p_seg) - 1) < 1e-10)

## -----
## Function to generate one Monte Carlo data set
## -----

generate_lcrol_data <- function(N, J, beta0, beta1, beta2, p_seg) {

  ## --- explanatory variables ---
  x1 <- rnorm(N, mean = 0, sd = 1)           # continuous covariate
  x2 <- rbinom(N, size = 1, prob = 0.5)      # 0/1 dummy

  ## --- latent utilities U_ij = beta0_j + x1_i*beta1_j + x2_i*beta2_j + eps_ij ---
  ## eps_ij follows a EV1 distribution
  eps <- matrix(-log(-log(runif(N * J))), nrow = N, ncol = J)

  U <- matrix(NA_real_, nrow = N, ncol = J)
  for (j in 1:J) {
    U[, j] <- beta0[j] + x1 * beta1[j] + x2 * beta2[j] + eps[, j]
  }

  ## --- true ranking implied by the utilities (most preferred first) ---
  true_order <- t(apply(U, 1, function(u) order(u, decreasing = TRUE)))
  colnames(true_order) <- paste0("true_rank_pos", 1:J)

  ## --- assign each individual to a ranking-capability segment ---
  segment_labels <- 0:(J - 1)
  segment <- sample(segment_labels, size = N, replace = TRUE, prob = p_seg)

  ## --- construct the OBSERVED ranking for each individual ---

```

```

## The first k positions (k = segment) equal the true utility order;
## the remaining J-k positions are a random permutation of the
## remaining items (the respondent cannot rank these correctly).
observed_order <- matrix(NA_integer_, nrow = N, ncol = J)
for (i in 1:N) {
  k <- segment[i]
  ord_i <- true_order[i, ]
  if (k > 0) {
    observed_order[i, 1:k] <- ord_i[1:k]
  }
  if (k < J) {
    remaining_items <- ord_i[(k + 1):J]
    if (length(remaining_items) > 1) {
      remaining_items <- sample(remaining_items) # random ordering
    }
    observed_order[i, (k + 1):J] <- remaining_items
  }
}
colnames(observed_order) <- paste0("obs_rank_pos", 1:J)

## --- also provide the "rank given to each item" representation ---
## y_ij = rank that individual i gives to item j (1 = most preferred)
y <- matrix(NA_integer_, nrow = N, ncol = J)
for (i in 1:N) {
  for (pos in 1:J) {
    item <- observed_order[i, pos]
    y[i, item] <- pos
  }
}
colnames(y) <- paste0("rank_item", 1:J)

data.frame(
  id = 1:N,
  x1 = x1,
  x2 = x2,
  segment = segment,
  U, # latent utilities (unobserved in practice)
  true_order, # true preference order (for checking purposes)
  observed_order, # observed order (survey response, item numbers)
  y # observed rank given to each item
)
}

## -----
## Generate one Monte Carlo replication
## -----

sim_data <- generate_lcrowl_data(N, J, beta0, beta1, beta2, p_seg)

## =====
## Function: rol_probability
##
## Computes  $P(r_i; \beta)$  for every individual  $i$ , where  $r_i = (r_{i1}, \dots, r_{iJ})$ 

```

```

## is the vector giving the item number that received rank j (rank 1 =
## most preferred),
##
##  $P(r_i; \beta) = \prod_{j=1}^{J-1} \exp(V_{i,r_{ij}}) / \sum_{l=1}^J \exp(V_{i,r_{il}})$ 
## =====

rol_probability <- function(data, beta_vec, J = NULL) {

  if (is.null(J)) {
    J <- sum(grepl("^obs_rank_pos", names(data)))
  }

  N <- nrow(data)

  if (length(beta_vec) != 3 * (J - 1)) {
    stop(sprintf("beta_vec must have length 3*(J-1) = %d, got %d",
                 3 * (J - 1), length(beta_vec)))
  }

  beta_mat <- matrix(beta_vec, nrow = J - 1, ncol = 3, byrow = TRUE)
  beta_mat <- rbind(beta_mat, c(0, 0, 0)) # alternative J = base category
  colnames(beta_mat) <- c("beta0", "beta1", "beta2")

  X <- cbind(1, data$x1, data$x2)
  V <- X %*% t(beta_mat)

  obs_cols <- paste0("obs_rank_pos", 1:J)
  order_mat <- as.matrix(data[, obs_cols])

  V_ordered <- matrix(NA_real_, nrow = N, ncol = J)
  for (i in 1:N) {
    V_ordered[i, ] <- V[i, order_mat[i, ]]
  }

  expV <- exp(V_ordered)
  denom <- t(apply(expV, 1, function(row) rev(cumsum(rev(row)))))

  ratio <- expV[, 1:(J - 1), drop = FALSE] / denom[, 1:(J - 1), drop = FALSE]
  P <- apply(ratio, 1, prod)

  return(P)
}

## quick check using the true parameter values from Table I
true_beta_vec <- c(1.00, 0.75, -0.30, # beta_1
                  0.25, -0.50, 0.45, # beta_2
                  -0.25, 1.00, 0.80) # beta_3

P_true <- rol_probability(sim_data, true_beta_vec)
summary(P_true) ; length(P_true)

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.001136 0.030513 0.065386 0.078370 0.101546 0.446267

```

```
## [1] 1000

## =====
## Function: rol_loglik
##
## ell(beta) = sum_i sum_{j=1}^{J-1} x'_{i,r_ij} beta
##             - sum_i sum_{j=1}^{J-1} log( sum_{l=j}^J exp(x'_{i,r_il} beta) )
## =====

rol_loglik <- function(data, beta_vec, J = NULL) {

  if (is.null(J)) {
    J <- sum(grepl("^obs_rank_pos", names(data)))
  }

  N <- nrow(data)

  if (length(beta_vec) != 3 * (J - 1)) {
    stop(sprintf("beta_vec must have length 3*(J-1) = %d, got %d",
                 3 * (J - 1), length(beta_vec)))
  }

  beta_mat <- matrix(beta_vec, nrow = J - 1, ncol = 3, byrow = TRUE)
  beta_mat <- rbind(beta_mat, c(0, 0, 0))

  X <- cbind(1, data$x1, data$x2)
  V <- X %*% t(beta_mat)

  obs_cols <- paste0("obs_rank_pos", 1:J)
  order_mat <- as.matrix(data[, obs_cols])

  V_ordered <- matrix(NA_real_, nrow = N, ncol = J)
  for (i in 1:N) {
    V_ordered[i, ] <- V[i, order_mat[i, ]]
  }

  expV <- exp(V_ordered)
  denom <- t(apply(expV, 1, function(row) rev(cumsum(rev(row)))))

  term1 <- sum(V_ordered[, 1:(J - 1), drop = FALSE])
  term2 <- sum(log(denom[, 1:(J - 1), drop = FALSE]))

  ell <- term1 - term2
  return(ell)
}

## quick checks
ll_true <- rol_loglik(sim_data, true_beta_vec) ; ll_true

## [1] -2895.989

## cross-check against rol_probability()
all.equal(ll_true, sum(log(rol_probability(sim_data, true_beta_vec)))) # OK

## [1] TRUE
```

```

## log-likelihood at beta = 0 should be worse (lower) than at the truth
rol_loglik(sim_data, rep(0, 9)) #OK

## [1] -3178.054

## =====
## Function: rol_gradient
##
## grad(beta) = sum_i sum_{j=1}^{J-1} ( x_{i,r_ij} - sum_{l=j}^J P_{i,r_il|j} x_{i,r_il} )
## =====

rol_gradient <- function(data, beta_vec, J = NULL) {

  if (is.null(J)) {
    J <- sum(grepl("^obs_rank_pos", names(data)))
  }

  N <- nrow(data)
  K <- 3 * (J - 1)

  if (length(beta_vec) != K) {
    stop(sprintf("beta_vec must have length 3*(J-1) = %d, got %d", K, length(beta_vec)))
  }

  beta_mat <- matrix(beta_vec, nrow = J - 1, ncol = 3, byrow = TRUE)
  beta_mat <- rbind(beta_mat, c(0, 0, 0))

  X <- cbind(1, data$x1, data$x2)
  V <- X %*% t(beta_mat)

  obs_cols <- paste0("obs_rank_pos", 1:J)
  order_mat <- as.matrix(data[, obs_cols])

  get_x_alt <- function(xi, alt) {
    v <- numeric(K)
    if (alt <= (J - 1)) {
      idx <- ((alt - 1) * 3 + 1):((alt - 1) * 3 + 3)
      v[idx] <- xi
    }
    v
  }

  grad <- numeric(K)

  for (i in 1:N) {

    r_i <- order_mat[i, ]
    xi <- X[i, ]
    V_i <- V[i, ]

    V_ordered <- V_i[r_i]
    expV_ordered <- exp(V_ordered)

    for (j in 1:(J - 1)) {

```

```

    idx_remaining <- j:J
    denom_j      <- sum(expV_ordered[idx_remaining])
    P_l_given_j  <- expV_ordered[idx_remaining] / denom_j

    x_rij <- get_x_alt(xi, r_i[j])

    x_weighted_sum <- numeric(K)
    for (m in seq_along(idx_remaining)) {
      l      <- idx_remaining[m]
      alt_l  <- r_i[l]
      x_weighted_sum <- x_weighted_sum + P_l_given_j[m] * get_x_alt(xi, alt_l)
    }

    grad <- grad + (x_rij - x_weighted_sum)
  }
}

return(grad)
}

## quick check: analytical gradient vs numerical gradient of rol_loglik()
library(numDeriv)
g_analytical <- rol_gradient(sim_data, true_beta_vec)
g_analytical

## [1] -65.028066 -39.306366 -48.635422 -5.340174 76.900307 -2.443560 69.712403
## [8] -55.642715 43.351263

g_numerical <- grad(function(b) rol_loglik(sim_data, b), true_beta_vec)
cbind(analytical = g_analytical, numerical = g_numerical)

##      analytical numerical
## [1,] -65.028066 -65.028066
## [2,] -39.306366 -39.306366
## [3,] -48.635422 -48.635423
## [4,] -5.340174 -5.340173
## [5,] 76.900307 76.900308
## [6,] -2.443560 -2.443561
## [7,] 69.712403 69.712404
## [8,] -55.642715 -55.642715
## [9,] 43.351263 43.351263

round(max(abs(g_analytical - g_numerical)), 5) # close to 0, OK.

## [1] 0

## =====
## Function: rol_hessian
##
## 
$$H(\beta) = \sum_i \sum_{\{j=1\}^{\{J-1\}}} \sum_{\{l=j\}^{\{J\}}} P_{\{i, r_{il}|j\}} * \\
## \quad (x_{\{i, r_{il}\}} - \sum_{\{k=j\}^{\{J\}}} x_{\{i, r_{ik}\}} P_{\{i, r_{ik}|j\}}) x'_{\{i, r_{il}\}}$$

## =====

rol_hessian <- function(data, beta_vec, J = NULL) {

```

```

if (is.null(J)) {
  J <- sum(grepl("^obs_rank_pos", names(data)))
}

N <- nrow(data)
K <- 3 * (J - 1)

if (length(beta_vec) != K) {
  stop(sprintf("beta_vec must have length 3*(J-1) = %d, got %d", K, length(beta_vec)))
}

beta_mat <- matrix(beta_vec, nrow = J - 1, ncol = 3, byrow = TRUE)
beta_mat <- rbind(beta_mat, c(0, 0, 0))

X <- cbind(1, data$x1, data$x2)
V <- X %*% t(beta_mat)

obs_cols <- paste0("obs_rank_pos", 1:J)
order_mat <- as.matrix(data[, obs_cols])

get_x_alt <- function(xi, alt) {
  v <- numeric(K)
  if (alt <= (J - 1)) {
    idx <- ((alt - 1) * 3 + 1):((alt - 1) * 3 + 3)
    v[idx] <- xi
  }
  v
}

H <- matrix(0, K, K)

for (i in 1:N) {
  r_i <- order_mat[i, ]
  xi <- X[i, ]
  V_i <- V[i, ]

  V_ordered <- V_i[r_i]
  expV_ordered <- exp(V_ordered)

  x_ordered <- lapply(r_i, function(alt) get_x_alt(xi, alt))

  for (j in 1:(J - 1)) {
    idx_remaining <- j:J
    denom_j <- sum(expV_ordered[idx_remaining])
    P_l_given_j <- expV_ordered[idx_remaining] / denom_j

    xbar_j <- numeric(K)
    for (m in seq_along(idx_remaining)) {
      xbar_j <- xbar_j + P_l_given_j[m] * x_ordered[[idx_remaining[m]]]
    }
  }
}

```

```

    for (m in seq_along(idx_remaining)) {
      l <- idx_remaining[m]
      x_l <- x_ordered[[l]]
      H <- H + P_l_given_j[m] * outer(x_l - xbar_j, x_l)
    }
  }
}

return(H)
}

## quick check: analytical Hessian vs numerical Hessian of rol_loglik()
H_analytical <- rol_hessian(sim_data, true_beta_vec)
H_numerical <- -hessian(function(b) rol_loglik(sim_data, b), true_beta_vec)

max(abs(H_analytical - H_numerical)) # OK

## [1] 2.448219e-06

isSymmetric(H_analytical) # OK

## [1] TRUE

## =====
## Function: rol_newton_raphson
##
## Newton-Raphson maximum likelihood estimation of the ROL model, using
## rol_loglik(), rol_gradient() and rol_hessian() defined above.
##
## IMPORTANT SIGN NOTE:  $H(\beta) = -\text{grad}_{\{\beta \beta'\}} \ell(\beta)$  is the
## NEGATIVE Hessian of the log-likelihood (positive semi-definite, since
##  $\ell$  is globally concave). The Newton step that maximizes  $\ell$  is
## therefore  $\beta_{\text{new}} = \beta + [H(\beta)]^{-1} g(\beta)$ , i.e. a PLUS, not
## a minus, when using  $H$  as defined here.
## =====

rol_newton_raphson <- function(data, start, J = NULL, tol = 1e-10,
                              maxit = 100, verbose = TRUE) {

  if (is.null(J)) {
    J <- sum(grepl("^obs_rank_pos", names(data)))
  }

  beta <- start

  for (it in 1:maxit) {

    g <- rol_gradient(data, beta, J)
    H <- rol_hessian(data, beta, J)

    step <- solve(H, g) #  $H^{-1} g$ 
    beta_new <- beta + step # PLUS, since  $H = -\text{Hessian}(\ell)$ 

    if (max(abs(beta_new - beta)) < tol) {
      beta <- beta_new
    }
  }
}

```

```

    if (verbose) cat("Converged in", it, "iterations\n")
    break
  }
  beta <- beta_new
}

ll <- rol_loglik(data, beta, J)

list(beta = beta, loglik = ll, Hessian = H, iterations = it)
}

## -----
## Example usage: fit the ROL model on a single replication (sim_data)
## -----

K <- 3 * (J - 1)

## descriptive names for the parameter vector: (intercept, x1, x2) per
## non-base alternative, matching the order used throughout
par_names <- paste0(rep(c("intercept_alt", "x1_alt", "x2_alt"), J - 1),
                     rep(1:(J - 1), each = 3))

start <- rep(0, K)
names(start) <- par_names

fit_nr <- rol_newton_raphson(sim_data, start)

## Converged in 5 iterations

names(fit_nr$beta) <- par_names
fit_nr

## $beta
## intercept_alt1      x1_alt1      x2_alt1 intercept_alt2      x1_alt2
##      0.9230319      0.6028879     -0.4254772      0.2625532     -0.3474106
##      x2_alt2 intercept_alt3      x1_alt3      x2_alt3
##      0.3738945     -0.1104752      0.7776769      0.7636815
##
## $loglik
## [1] -2870.613
##
## $Hessian
##           [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
## [1,]  425.37542  -19.218965  220.125540  -140.502761    9.98994980  -69.67959361
## [2,]  -19.21897  403.814963   9.570740    9.989950 -108.90191041  -6.29734148
## [3,]  220.12554   9.570740  220.125540  -69.679594  -6.29734148  -69.67959361
## [4,]  -140.50276   9.989950  -69.679594  430.411060  11.01432636  211.69448189
## [5,]    9.98995 -108.901910  -6.297341   11.014326  342.81366142  16.65475893
## [6,]  -69.67959  -6.297341  -69.679594  211.694482  16.65475893  211.69448189
## [7,] -149.16414  -11.471713  -83.102589  -134.440770  -8.73974119  -67.23685293
## [8,]  -11.47171 -160.827360  -6.073337  -8.739741  -84.63864182  -0.02194264
## [9,]  -83.10259  -6.073337  -83.102589  -67.236853  -0.02194264  -67.23685293
##           [,7]      [,8]      [,9]
## [1,] -149.164143  -11.47171290  -83.10258911

```

```
## [2,] -11.471713 -160.82736035 -6.07333723
## [3,] -83.102589 -6.07333723 -83.10258911
## [4,] -134.440770 -8.73974119 -67.23685293
## [5,] -8.739741 -84.63864182 -0.02194264
## [6,] -67.236853 -0.02194264 -67.23685293
## [7,] 425.807895 13.49739239 217.07240538
## [8,] 13.497392 358.89848597 -5.03640515
## [9,] 217.072405 -5.03640515 217.07240538
##
## $iterations
## [1] 5

se_nr <- sqrt(diag(solve(fit_nr$Hessian)))
names(se_nr) <- par_names
se_nr

## intercept_alt1      x1_alt1      x2_alt1 intercept_alt2      x1_alt2
## 0.08465257      0.06185411      0.11950307      0.08137661      0.06239666
##      x2_alt2 intercept_alt3      x1_alt3      x2_alt3
## 0.11691463      0.08298425      0.06445983      0.11867129

## quick comparison against the true beta values used to simulate the data
cbind(estimate = fit_nr$beta, true = true_beta_vec, se = se_nr)

##      estimate      true      se
## intercept_alt1 0.9230319 1.00 0.08465257
## x1_alt1      0.6028879 0.75 0.06185411
## x2_alt1      -0.4254772 -0.30 0.11950307
## intercept_alt2 0.2625532 0.25 0.08137661
## x1_alt2      -0.3474106 -0.50 0.06239666
## x2_alt2      0.3738945 0.45 0.11691463
## intercept_alt3 -0.1104752 -0.25 0.08298425
## x1_alt3      0.7776769 1.00 0.06445983
## x2_alt3      0.7636815 0.80 0.11867129

## =====
## Monte Carlo study: 10,000 replications
##
## For each replication we (i) simulate a fresh data set with
## generate_lrcol_data(), (ii) estimate beta via rol_newton_raphson()
## (starting at 0 each time), and (iii) store the resulting estimates.
## At the end we report, for each parameter, the mean and standard
## deviation of the estimates across replications, together with the
## true value used to generate the data -- mirroring Table I of the paper.
##
## NOTE: this loop takes a long time to run (rol_gradient() and
## rol_hessian() are not fully vectorized), so it is NOT executed when
## knitting this document. It is kept below, commented out, purely for
## documentation of how the results were produced. The resulting summary
## table was computed once, saved to disk with
## save(mc_summary, file = "mc_summary_ROL"), and is simply loaded and
## printed below.
## =====

# n_reps <- 10000
#
```

```

# mc_results <- matrix(NA_real_, nrow = n_reps, ncol = K)
# colnames(mc_results) <- par_names
#
# set.seed(1234) # separate seed dedicated to the Monte Carlo study
#
# t0 <- proc.time()
#
# for (r in 1:n_reps) {
#
#   dat_r <- generate_lcrol_data(N, J, beta0, beta1, beta2, p_seg)
#
#   fit_r <- tryCatch(
#     rol_newton_raphson(dat_r, start, verbose = FALSE),
#     error = function(e) NULL
#   )
#
#   if (!is.null(fit_r)) {
#     mc_results[r, ] <- fit_r$beta
#   }
#
#   if (r %% 500 == 0) {
#     cat("Replication", r, "of", n_reps,
#         "-- elapsed:", round((proc.time() - t0)["elapsed"], 1), "sec\n")
#   }
# }
#
# cat("Total time:", round((proc.time() - t0)["elapsed"] / 60, 1), "minutes\n")
#
# n_failed <- sum(is.na(mc_results[, 1]))
# cat("Number of failed replications (dropped):", n_failed, "\n")
#
# mc_mean <- colMeans(mc_results, na.rm = TRUE)
# mc_sd <- apply(mc_results, 2, sd, na.rm = TRUE)
#
# mc_summary <- data.frame(
#   parameter = par_names,
#   true      = true_beta_vec,
#   mc_mean   = mc_mean,
#   mc_sd     = mc_sd,
#   bias      = mc_mean - true_beta_vec
# )
#
# print(mc_summary, digits = 4)
#
# save(mc_summary, file = "mc_summary_ROL")

## -----
## Load the previously computed Monte Carlo summary (10,000 replications)
## and print it, instead of re-running the loop above.
## -----

load("mc_summary_ROL")
print(mc_summary, digits = 4)

```

##		parameter	true	mc_mean	mc_sd	bias
##	intercept_alt1	intercept_alt1	1.00	0.8047	0.09042	-0.19526
##	x1_alt1	x1_alt1	0.75	0.5391	0.06434	-0.21093
##	x2_alt1	x2_alt1	-0.30	-0.3245	0.12213	-0.02447
##	intercept_alt2	intercept_alt2	0.25	0.2384	0.08329	-0.01157
##	x1_alt2	x1_alt2	-0.50	-0.4161	0.06322	0.08387
##	x2_alt2	x2_alt2	0.45	0.3111	0.11707	-0.13891
##	intercept_alt3	intercept_alt3	-0.25	-0.1064	0.08168	0.14356
##	x1_alt3	x1_alt3	1.00	0.6177	0.06353	-0.38231
##	x2_alt3	x2_alt3	0.80	0.5061	0.11470	-0.29389

Limitations of the ROL model

As depicted in the Monte Carlo simulation, in the context where respondents are not able to rank all the items according to their true preferences, the ROL model fail to estimate consistently the parameters of interest. The reason being that the model specifically assumes that these respondents are able to rank each of the items according to their true preferences. Ways of dealing with this issue are discussed in For, Paap, and Van Dijk (2010). We will discuss them later on.